TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 7: Practical Exercises

Foreign Trade University

Instructions

For each problem below there is a starter file `exercises/starter_code_problemNN.py` containing function signatures with `raise NotImplementedError` bodies, and a small `__main__` block with tests. Implement the missing functions so the tests pass. A reference solution is provided in `solutions/solution_problemNN.py` – try the problem yourself before looking!

All problems use only the Python 3.10+ standard library (including `math` and `cmath` for the FFT). Shared helper functions live in `starter_code.py` at the week root: `generate_points`, `generate_distinct_points`, `euclidean`, `brute_force_closest`, `poly_eval`, `poly_mul_naive`, `next_power_of_two` and the `Counter` instrumentation helper.

The 12 problems are grouped into three themes:

- **Closest Pair** (Problems 1–3): the brute-force $O(n^2)$ algorithm, the $O(n \log n)$ divide-and-conquer algorithm with the strip argument, and a fast-vs-slow verification harness.
- **Multiplication & Convolution** (Problems 4–5, 9–12): Karatsuba integer multiplication, naive polynomial multiplication, Strassen's 7-multiplication matrix product, counting range sums by divide and conquer, median of two sorted arrays, and fast modular exponentiation.
- **FFT** (Problems 6–8): the recursive radix-2 FFT and its inverse, polynomial multiplication via the FFT, and linear convolution / cross-correlation via the FFT.

1 Closest Pair

Problem 1 – Brute-Force Closest Pair of Points

Implement `closest_pair_brute(points)` that returns the minimum Euclidean distance between any two of the given 2-D points, together with the achieving pair, by checking all $\binom{n}{2}$ pairs in $O(n^2)$. Return `(min_distance, (p, q))`, and raise `ValueError` for fewer than two points. Use `euclidean` from `starter_code.py`.

Example. `closest_pair_brute([(0,0),(3,4),(1,1)])` returns distance $\sqrt{2}$ with the pair $\{(0,0),(1,1)\}$.

Problem 2 – Divide-and-Conquer Closest Pair $O(n \log n)$

Implement `closest_pair_dc(points)`, the classic divide-and-conquer algorithm:

```
Sort points by x (px) and by y (py).
Split px at the median x into left/right; split py accordingly.
Recurse on each half; let delta = min of the two minimum distances.
Build the "strip" of points within delta of the dividing line, in y-order.
Compare each strip point only with the next 7 in y-order; update delta.
Return (min_distance, (p, q)).
```

Raise `ValueError` for fewer than two points. The strip argument (the 7/15-neighbour lemma) is what makes the combine step linear, giving the recurrence $T(n) = 2T(n/2) + O(n) = O(n \log n)$.

Problem 3 – Verify D&C Closest Pair Equals Brute Force

Implement `verify_closest_pair(num_trials, n, seed)`: for each of `num_trials` random instances of `n` distinct points (`generate_distinct_points(n, seed=seed + trial)`), compute the minimum distance from the divide-and-conquer algorithm (Problem 2) and from `brute_force_closest` (`starter_code.py`). Return `True` iff they agree on every trial – the standard empirical correctness check.

2 Multiplication & Convolution

Problem 4 – Karatsuba Integer Multiplication

Implement `karatsuba(x, y, counter=None)` for non-negative integers using exactly **three** recursive multiplications per level:

```
Split x = x_high * B + x_low, y = y_high * B + y_low  (B = 10^half).
z0 = karatsuba(x_low,  y_low)
z2 = karatsuba(x_high, y_high)
z1 = karatsuba(x_low + x_high, y_low + y_high) - z0 - z2
return z2 * B^2 + z1 * B + z0
```

Use a single-digit base case; if a `Counter` is supplied, `tick()` once per base-case multiplication. Raise `ValueError` on negative inputs. The recurrence is $T(n) = 3T(n/2) + O(n) = \Theta(n^{\log_2 3})$.

Problem 5 – Naive Polynomial Multiplication (Coefficient Convolution)

Implement `poly_mul_naive(a, b)` that multiplies two polynomials given as coefficient lists in increasing degree order (`[a0, a1, a2]` means $a_0 + a_1x + a_2x^2$), in $O(nm)$ via the convolution $(a * b)[k] = \sum_{i+j=k} a_i b_j$. The product has $n + m - 1$ coefficients; return `[]` if either input is empty.

Example. `poly_mul_naive([1,2,3],[4,5]) == [4,13,22,15]`.

Problem 9 – Strassen's Matrix Multiplication: 7 vs 8 Mults

Implement `naive_mult_count(n)` ($T(n) = 8T(n/2)$, base case 1) and `strassen_mult_count(n)` ($T(n) = 7T(n/2)$, base case 1), the scalar-multiplication counts for the two recurrences. Then implement `strassen_2x2(A, B, counter=None)` that multiplies two 2×2 matrices using Strassen's seven products (ticking 7 if a `Counter` is given), and verify it matches the provided `naive_2x2` on random matrices. Confirm `strassen_mult_count`$(n) <$ `naive_mult_count`(n) for $n \geq 2$.

Problem 10 – Count of Range Sums via Divide and Conquer

Given an integer array `nums` and bounds $\ell \leq u$, count the contiguous subarrays whose sum lies in $[\ell, u]$. Implement `count_range_sum_dc(nums, lower, upper)` in $O(n \log n)$ by mergesorting the prefix-sum array and, during each merge, using a two-pointer window to count for each left index the number of right indices j with $\ell \leq S[j] - S[i] \leq u$. Verify against the provided brute-force baseline.

Example. `count_range_sum_dc([-2,5,-1], -2, 2) == 3`.

Problem 11 – Median of Two Sorted Arrays in $O(\log(m+n))$

Implement `median_two_sorted(a, b)` for two sorted arrays (at least one non-empty), in $O(\log(\min(m, n)))$, by binary-searching the partition of the shorter array so that everything on the left is $\leq$ everything on the right (`a_left <= b_right and b_left <= a_right`, using $\pm\infty$ sentinels). Verify against the provided merge-based reference.

Example. `median_two_sorted([1,2],[3,4]) == 2.5`.

Problem 12 – Fast Modular Exponentiation (Square-and-Multiply)

Implement `power_mod(base, exp, mod, counter=None)` computing $(\text{base}^{\text{exp}})$ mod mod in $O(\log \text{exp})$ modular multiplications via square-and-multiply. `exp` must be non-negative and `mod` positive; if a `Counter` is given, `tick()` once per modular multiplication. Verify against Python's built-in `pow(base, exp, mod)`.

3 FFT

Problem 6 – Recursive Radix-2 FFT and Inverse FFT

Implement the recursive radix-2 Cooley–Tukey FFT and its inverse:

```
fft(a):  pad a to the next power of two; return the DFT as complex values.
         Split into even/odd indices, recurse, combine with butterflies
         using roots of unity omega_n^k = exp(-2*pi*i*k/n).
ifft(a): len(a) must be a power of two; same recursion with the conjugate
         root exp(+2*pi*i*k/n), then divide every entry by n, so that
         ifft(fft(x)) recovers x (after zero-padding).
```

Use `cmath`/`math` (both standard library) and `next_power_of_two` from `starter_code.py`.

Problem 7 – Polynomial Multiplication via FFT

Implement `poly_mul_fft(a, b)` that multiplies two integer-coefficient polynomials (low-to-high) in $O(n \log n)$: let $L = \texttt{len}(a) + \texttt{len}(b) - 1$ and $\texttt{size} = \texttt{next\_power\_of\_two}(L)$; zero-pad and FFT both (Problem 6); multiply the spectra pointwise; inverse-FFT; round the first L entries to integers. Return `[]` for empty input. Verify it matches `poly_mul_naive`.

Problem 8 – Linear Convolution / Cross-Correlation via FFT

Linear convolution of two integer sequences is exactly polynomial multiplication. Implement `convolve_fft(a, b)` (delegate to Problem 7) and `cross_correlation(a, b)` (the convolution of `a` with the reversed `b`). Verify both against the provided direct $O(nm)$ definitions `convolve_direct` and `cross_correlation_direct`.

4 LeetCode Practice

After completing the problems above, practise this week's divide-and-conquer techniques (binary-search divide and conquer, merge-based counting, big-integer multiplication, fast exponentiation) on the following LeetCode problems. Difficulty is LeetCode's own label.

- Median of Two Sorted Arrays – *Hard* – binary search D&C.
- Search a 2D Matrix II – *Medium* – divide & conquer.
- Kth Smallest Element in a Sorted Matrix – *Medium* – binary search on answer.
- Find K-th Smallest Pair Distance – *Hard* – binary search + sorting.
- The Skyline Problem – *Hard* – divide & conquer.
- Count of Range Sum – *Hard* – merge sort.
- Multiply Strings – *Medium* – big-integer multiplication.
- Super Pow – *Medium* – fast exponentiation.